DECKWATCH KENYA

DESIGN NOTE · FEATURE: MONTHLY SAFETY REPORT

The monthly safety report
decisions, plan and how it works

Why Deckwatch now publishes a monthly report, the choices behind how it is built, how a report gets from news feeds to the dashboard, and what is left to do.

STATUS	COMMIT	DATE
Built and merged to main	**947ffdb**	**1 October 2026**

IN ONE PARAGRAPH

On the last day of every month, a scheduled job takes every article Deckwatch captured that month and asks Claude to group them into distinct real incidents, correct their counties, set aside anything that isn't a new incident, and write the report's editorial notes. Python code then counts everything from the articles themselves and stores the finished report. The app shows it at `/dashboard/reports/<month>`, with a card at the top of the dashboard. September 2026, reviewed by hand, is the first issue. Three production steps remain before October's report runs on its own on 31 October.

1 Why a monthly report
2 Key decisions
3 How it works, end to end
4 The report document
5 What the reader sees
6 Schedule and timing
7 Running and fixing it
8 Plan: done, next, later

9 Risks and limitations

10 Where the code lives

Why a monthly report

The live map answers "what is happening near me right now". It can't answer "what happened in Kenya this month, where, and what changed". That is the question a monthly report answers for someone deciding how to travel, commute, attend an event or send someone somewhere.

Building the first report by hand showed that the raw feed can't be published as-is. The feed stores **articles**, and many articles often describe one incident. September's numbers made the gap concrete:

369 articles captured in September	**105** distinct incidents after review	**10** duplicates merged by the live pipeline	**23×** the Kajiado aircraft crash, counted as separate incidents
41 off-topic items, such as a TikTok captions explainer filed as a missing person	**116** context items: flood warnings, outage notices, court cases over older crimes	**~17%** of incident articles placed in the wrong county	**361** articles still marked "unconfirmed" by automatic scoring

Published raw, the report would have overstated the month by about 3.5 times, blamed Nairobi for a Turkana border gunfight, and hidden the Isiolo–Samburu banditry cluster, the month's deadliest, under crime articles from elsewhere. The report exists to do that consolidation every month, and to be honest about what it still doesn't know.

> **Editorial stance.** The report says what happened, where, how confident we are and what changed. It counts incidents *captured by Deckwatch*, never calls a county "most dangerous", gives death tolls as the lowest figure any source reported, and links every incident to the articles it came from.

Key decisions

Each decision below records what was chosen, why, and what was considered instead.

D1

Count incidents, not articles

DECISION	Every report consolidates the month's articles into distinct incidents before counting anything. Articles that aren't a new incident become *context* or *off-topic*, and both counts are shown.
WHY	The live pipeline merged only 10 duplicates in September. Raw counts reflect media attention, not events.
INSTEAD	Fixing deduplication in the live pipeline first. That is still worth doing (see §8), but it would have delayed the report, and a monthly batch can afford a slower, more careful pass than a 15-minute ingest.

D2

Claude does the editorial pass. Code does the counting.

DECISION	One request a month to `claude-opus-5-5` returns the grouping, corrected counties and categories, death floors, hotspots, "why it matters" notes, the watch list and two headlines. Python then derives every published number from the articles (totals, outlet counts, agreement rates, coverage dates).
WHY	Grouping news reports by event is a judgement task that rules can't do well. Counting is not, and letting a model produce figures would make them unverifiable. Output is forced into a JSON schema, and county and category are restricted to fixed lists.
INSTEAD	Rule-based clustering by title similarity and date, which fails on follow-ups ("victims named", "death toll rises"). A smaller model was also considered; the most capable one was chosen because this runs once a month, so quality matters more than cost (estimated at roughly US$1 a month, not yet measured).

D3

Distrust the model's output before storing it

DECISION	A cleaning step (`clean_consolidation`) drops article ids that don't exist, gives each article to only one group, drops empty groups, repairs unknown counties, out-of-month dates and negative deaths, and removes hotspots and "defining" picks that point at nothing.
WHY	A partly wrong answer should make a slightly worse report, not a broken page or an invented number.

D4

It lives under the Dashboard

DECISION — `/dashboard/reports` (archive) and `/dashboard/reports/2026-09` (a report), with a "Monthly safety report" card at the top of the National Dashboard.

WHY — The report is national intelligence, the same job as the dashboard, and the Dashboard tab stays highlighted while reading it. `/reports` was already taken by citizen report submission.

INSTEAD — A top-level nav item. Rejected for now, because one report a month doesn't need a permanent tab.

D5

One stored document per month

DECISION — A new `monthly_reports` table keyed by month (`"2026-09"`) holds the whole report as one JSON document. Regenerating a month replaces it.

WHY — A report is written once and read whole, so there is nothing to gain from splitting it into tables. A stored report also stays stable after live data changes.

D6

Publish on the last day, 23:30 Kenya time, from GitHub Actions

DECISION — A workflow runs at 23:30 EAT on the 28th–31st and works out whether a month is actually due. Months are cut on the Kenyan calendar (EAT, UTC+3).

WHY — You asked for the last day. Ingestion already runs on Actions for free, and Render cron jobs require a paid plan. GitHub starts scheduled runs late, so the job also accepts the morning of the 1st, so a late run can't skip a month (details in §6).

INSTEAD — Running at 00:30 on the 1st, which would give a complete month. It was kept as the late-run fallback rather than the plan, to honour the last-day preference.

D7

Never fail silently

DECISION — Without an API key the job still stores an *automatic* report (one incident per article), and the page labels it "not reviewed". If the backend can't be reached, the app serves reports bundled with it.

WHY — A missing report on the 31st is worse than a clearly labelled rough one. The method is always shown: reviewed by hand, consolidated by Claude, or automatic and unreviewed.

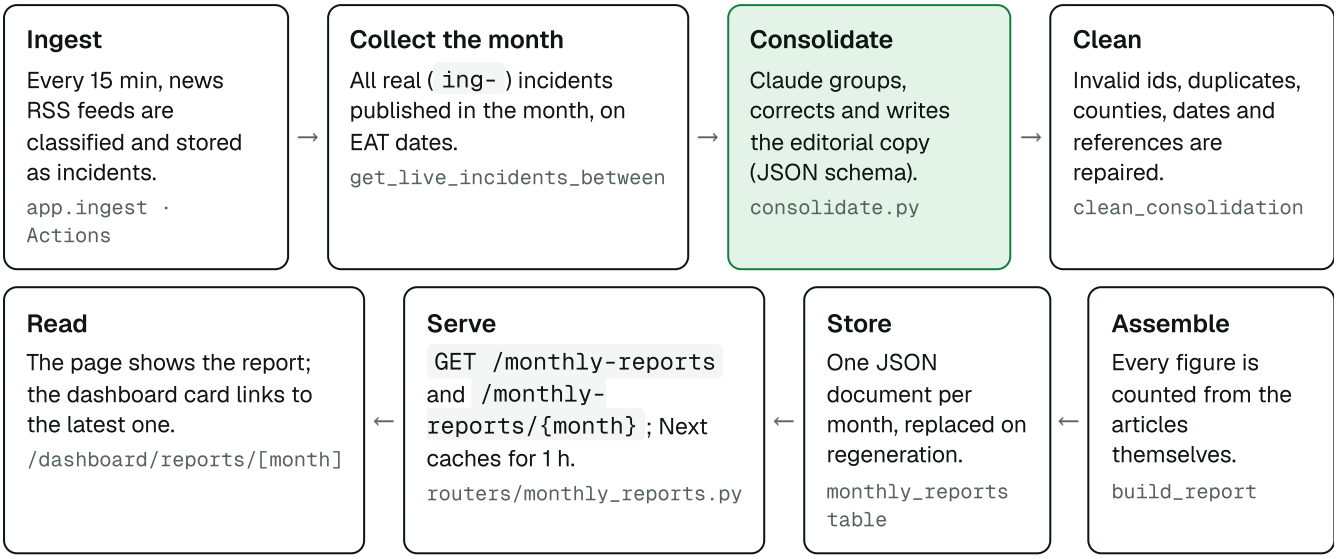
D8

September stays hand-reviewed

DECISION — The hand-built September consolidation was converted into the same document format by the same Python builder and ships with the app as the first issue.

WHY — It is the best version we have, and it gives a reference to check the automated October report against. The builder reproduced every hand-computed figure exactly.

How it works, end to end



Green: the only step that uses a model. Everything else is deterministic and covered by tests with no database or API key.

Step by step

1. **Collect.** The job reads every ingested incident whose publish time falls in the month's EAT window, from 00:00 on the 1st to 00:00 on the 1st of the next month. Seed and demo rows are excluded, because their timestamps are synthetic.
2. **Consolidate.** Each article goes to Claude as one line: short id, EAT date, outlets, the classifier's category and county, headline and summary. The system prompt encodes the editorial rules used for September, including what counts as one event, what is context, deaths as the lowest reported figure, and no "most dangerous" language. The request streams, uses adaptive thinking at high effort, and sets `fallbacks: "default"` so a safety-filter refusal is retried server-side on a fallback model instead of failing. A refusal or a truncated answer raises an error rather than storing half a report.
3. **Clean.** The answer is checked against the real article list and repaired (D3).
4. **Assemble.** The builder computes, per incident, the article count, distinct outlets and a de-duplicated source list. For the month it computes totals, off-topic and context counts, outlet volumes, verification-status counts, how often the classifier's category and county matched the review, and the first day of full coverage. That last figure is how September's late start (12 September) is detected and flagged automatically.
5. **Store and serve.** The document is upserted into `monthly_reports` . The Next.js server reads it from the backend (cached for an hour; the archive list for 10 minutes), falling back to reports bundled in `src/lib/data/monthly-reports/` .
6. **Render.** The page derives the rest itself: category and county breakdowns, the daily timeline, weekly blocks and, when last month's report exists, the month-on-month comparison.

4

The report document

One JSON document per month, built by `backend/app/monthly/report.py` and typed as `MonthlyReport` in `src/lib/monthly-reports.ts`. The two must change together, and `version` (currently 1) is bumped if the shape changes.

FIELD	WHAT IT HOLDS	SOURCE
`month` , `asOf` , `generatedAt`	The month, the data cut-off and when it was built	Job
`method` , `model`	*editorial*, *claude* or *automatic*, and the model used	Job
`coverageFrom`	First day of full coverage, or null when complete from the 1st	Computed
`articles` , `eventArticles` , `contextArticles` , `offTopic`	The funnel from captured articles to incidents. The three parts always add up to `articles` .	Computed
`autoMerged` , `statusCounts` , `catAgree` , `countyAgree`	How well the live pipeline did on its own, used in "what we don't know"	Computed
`outletCounts` , `context`	Articles per outlet; the labelled context groups and their sizes	Computed / model labels
`events[]`	Each incident: title, date, county, category, deaths, article and outlet counts, sources with links, and optionally hotspot, conflict note, missing-person status, defining flag and "why it matters"	Model grouping + computed counts
`hotspots[]`	Places, corridors or patterns with at least two member incidents	Model
`watch[]`	Next month's watch list: title, facts and one practical action	Model
`headlines`	Headlines for sections 01 and 02	Model

> **Report categories are coarser than the map's.** Nine reader-facing groups (Crime & security, Road & transport, Fire & explosions, Protests & unrest, Missing persons, Building & mine collapses, Flooding & weather, Animal & public health, Wildlife) replace the eleven classifier categories. A reader cares that something was a crime, not whether the classifier called it crime or terror alert.

5

What the reader sees

The page follows the ten-section outline agreed at the start, and every section is filled from the document, so no month-specific wording lives in the code.

SECTION	CONTENTS
Cover	Five headline figures, the method label, the data cut-off, and banners for partial coverage or an unreviewed report
01 The month in safety	Headline plus a lede generated from the numbers: totals, deadliest category, deadliest hotspot, most-reported story
02 What happened	Incidents and deaths by category
03 Where it happened	A county map shaded by incident count, with hover details, and a ranked table. Clicking either opens that county's brief. Counties with nothing captured are listed by name.
04 Emerging hotspots	Cards per hotspot: incidents, deaths, counties and the member incidents
05 What changed	Month-on-month by category once a previous report exists, plus a daily timeline with partial coverage shaded and week-by-week blocks
06 Defining incidents	5 to 8 cards: deaths, number of outlets, conflicting-report flags, why it matters, source links
07 Your county	A county picker with a rank, the top categories, the busiest day, open missing-person cases and every incident
08 Know / don't know	Corroboration, classifier accuracy, conflicts, coverage gaps; generated from the data
09 What to watch	Next month's items, each with a "Do" action
10 Sources and method	The article funnel, context groups, outlet volumes, and a filterable log of every incident
Help	999 · 112 · 911, Red Cross 1199, GBV line 1195, Childline 116

6

Schedule and timing

Cron can't express "the last day of the month", and GitHub often starts scheduled runs late, sometimes by more than half an hour. The workflow therefore fires at **20:30 UTC (23:30 EAT) on the 28th–31st**, and `month_due()` decides what, if anything, to publish:

WHEN THE RUN ACTUALLY STARTS (EAT)	RESULT	WHY
30 Sep 23:30 (on time)	**September**	The last day, after 18:00
1 Oct 00:40 (late)	**September**	The 1st before noon, so the run must not skip the month
31 Oct 00:40 (the 30th's run, late)	Nothing	Must not publish October a day early
28, 29 Sep 23:30	Nothing	Not the last day
28 Feb 2028 (leap year)	Nothing	The 29th's run handles it

On a scheduled run the workflow ingests first, so the day's last articles make it in, then generates and stores the report and keeps the JSON as a workflow artifact for 90 days. Runs never overlap. The last half hour of the month (23:30–24:00) isn't included. Re-run that month manually if a complete month matters more than publishing on the last day.

7

Running and fixing it

```
# Generate and store a month (needs DATABASE_URL; uses ANTHROPIC_API_KEY if set)
python -m app.monthly --month 2026-10

# Dry run: write the JSON, don't touch the database
python -m app.monthly --month 2026-10 --no-store --out report.json

# Work from a saved GET /incidents response instead of the database
python -m app.monthly --month 2026-09 --articles incidents.json --no-store --out r.json

# Store a hand-edited report (e.g. after correcting a grouping)
python -m app.monthly --import report.json
```

Regenerate from GitHub

Go to Actions, then Monthly safety report, then Run workflow, and enter the month. **This replaces the stored report**, including a hand-reviewed one. Leave the month empty unless that is what you want.

Fix a wrong grouping

Download the run's `report.json` artifact (or use `--out`), edit it, and store it with `--import`. The page shows whatever document is stored.

Configuration

SETTING	WHERE	PURPOSE
`DATABASE_URL`	Repo secret (already used by ingest)	Read incidents, store reports
`ANTHROPIC_API_KEY`	Repo secret	Consolidation. Without it the report is automatic and labelled unreviewed
`REPORT_MODEL`	Repo variable (optional)	Defaults to `claude-opus-5-5`

8

Plan: done, next, later

STATUS	ITEM	NOTES
DONE	Generator, storage, API, scheduled workflow, report pages, dashboard card, archive, changelog entry	Commit 947ffdb on main
DONE	September 2026 published as issue 01, bundled with the app	Served even before the database has it
DONE	Tests: 88 backend (the model is stubbed, no API key needed), 108 frontend; lint, typecheck and build pass	Verified after rebasing on 14 newer commits
NEXT	Deploy the backend. The `monthly_reports` table is created on startup.	Production change
NEXT	Import September into the production database with `--import`	Production write; needs your go-ahead
NEXT	Confirm the `ANTHROPIC_API_KEY` repository secret is set	Otherwise October is automatic
NEXT	Dry-run Claude on September and compare with the hand review, to validate the prompt before it runs unattended	Estimated under US$1
NEXT	31 October: first automatic report. Review it on 1 November, correct with `--import` if needed.	First month-on-month comparison
LATER	Feed the consolidation back into the live pipeline: cross-outlet deduplication and county correction, so the map improves too	The report shows what is missing
LATER	A review step before publishing (draft, then approve) if an unattended report proves too risky	Decide after October
LATER	County briefs as a subscription ("my county, every month"), plus a shareable image or PDF of the summary	From the original proposal
LATER	Connect official early warnings (Kenya Meteorological Department, NDMA bulletins) with reported incidents in "What to watch"	Needs new sources

9

Risks and limitations

- **The prompt hasn't been run for real yet.** Every test uses a stub. Until the September dry run, grouping quality is an assumption. The cleaning step limits the damage, but can't catch a plausible wrong merge.
- **Unattended publishing.** An automated report goes live without human review. Mitigations: the method label, source links on every incident, the saved JSON artifact and `--import` for corrections.
- **Coverage bias.** Twelve mostly Nairobi-based outlets and no citizen reports yet. Remote counties are under-counted. The report says so and names counties with nothing captured.

- **Incomplete final half hour.** It is the cost of publishing on the last day. It can be closed with a manual re-run.
- **Regeneration overwrites.** A manual run for a month replaces the stored report, including a hand-reviewed one. The workflow warns about this.
- **A missing month returns a "not found" page with status 200**, because the page streams before it knows the month is missing. Search engines are told not to index it.
- **Cost** is estimated at about US$1 a month for one Opus request over ~370 articles. It grows with the number of articles captured, and should be measured on the first real run.

10

Where the code lives

PATH	ROLE
`backend/app/monthly/report.py`	Calendar rules, cleaning, assembly (pure, no DB)
`backend/app/monthly/consolidate.py`	Claude prompt, JSON schema and request
`backend/app/monthly/__main__.py`	Command line: generate, dry run, import
`backend/app/routers/monthly_reports.py`	`GET /monthly-reports`, `/monthly-reports/{month}`
`backend/app/models.py`, `repository.py`	`monthly_reports` table and queries
`.github/workflows/monthly-report.yml`	Last-day schedule and manual runs
`src/lib/monthly-reports.ts`	Types, data source with fallback, derivations
`src/app/dashboard/reports/`	Archive and report pages
`src/components/reports/`	Map, county panels, timeline, incident log
`src/lib/data/monthly-reports/2026-09.json`	September 2026, hand-reviewed
